

Problem 1

- Null model. Report the average train and test MSE of the null model that always predicts training $\bar{y}$ (average training y)

We do this in Python and obtain a train MSE of 0.8657 and a test MSE of 0.8658.

- OLS regression computed analytically by solving the following normal equations:

$$\left(\frac{1}{N}X^TX + \lambda I_p\right)\beta = \frac{1}{N}X^TY$$

where $\lambda \in \{0, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ and N is the number of observations in X . Report in a table the average training and test R^2 and MSE, as well as their standard deviations obtained from the 5 folds. On the same graph, plot the average training and test MSE vs λ as two separate curves. Also plot the average value of the logarithm of the determinant of $\frac{1}{N}X^TX + \lambda I_p$ vs λ .

We do this in Python and obtain table 1, figure 1, and figure 2

Lambda	Train R2	Test R2	Train MSE	Test MSE
0.000000	0.127 (0.001)	-0.001 (0.274)	0.756 (0.004)	0.867 (0.244)
0.000010	0.127 (0.001)	-0.063 (0.413)	0.756 (0.004)	0.922 (0.366)
0.000100	0.127 (0.001)	-0.202 (0.725)	0.756 (0.004)	1.044 (0.638)
0.001000	0.127 (0.001)	-0.035 (0.351)	0.756 (0.004)	0.897 (0.311)
0.010000	0.126 (0.001)	0.108 (0.029)	0.757 (0.004)	0.772 (0.033)
0.100000	0.118 (0.001)	0.114 (0.006)	0.763 (0.004)	0.767 (0.017)

Table 1: Mean (SD) of MSE and R^2 5 fold CV. Ridge-like Regression.

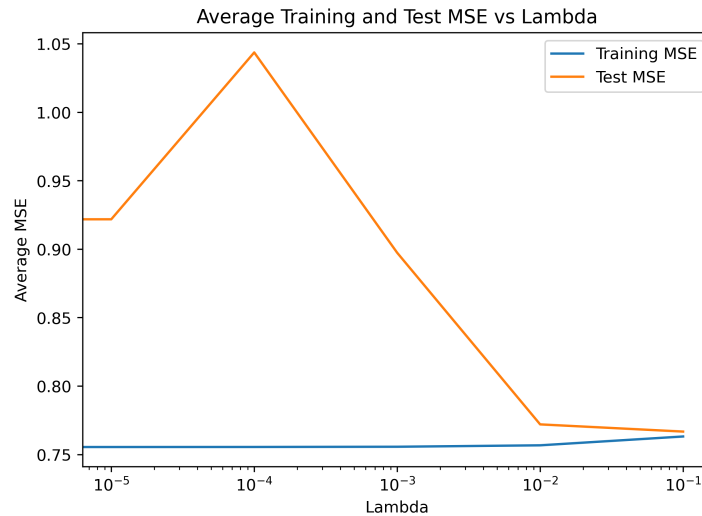


Figure 1: Average training and testing MSE vs Lambda.

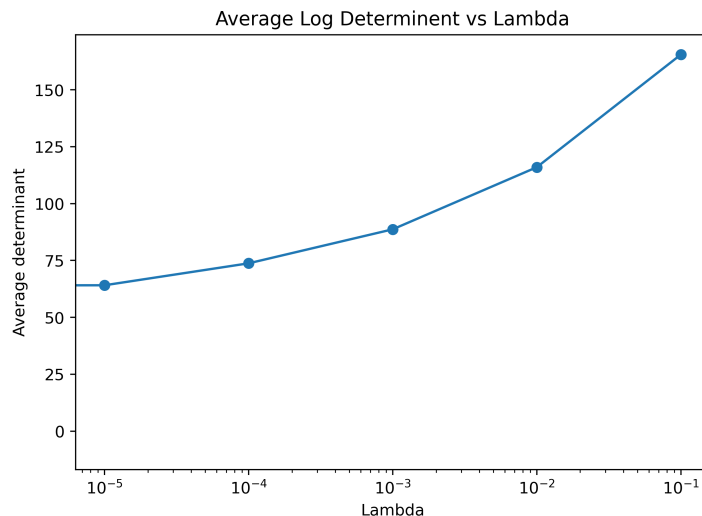


Figure 2: Average log determinant vs Lambda.

- OLS regression with backward elimination, for example using SequentialFeatureSelector. For each number of features $k \in \{5, 10, 15, \dots, 55\}$ report the average training and test R^2 and MSE in a table. On the same graph, plot the average training and test R^2 vs k as two separate cruves.

Doing this, we get table 2 and figure 3.

k	train R^2	test r2
5	0.0823	0.0806
10	0.1059	0.1032
15	0.1136	0.1112
20	0.1174	0.1137
25	0.1200	0.1156
30	0.1212	0.1164
35	0.1221	-1.6069
40	0.1226	-1.6933
45	0.1230	0.0895
50	0.1233	0.0562
55	0.1250	0.0877

Table 2: Average Training and Test R^2 for Backward Feature Selection

The negative test R^2 values are odd, but not impossible given what R^2 is measuring. The definition of R^2 is:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

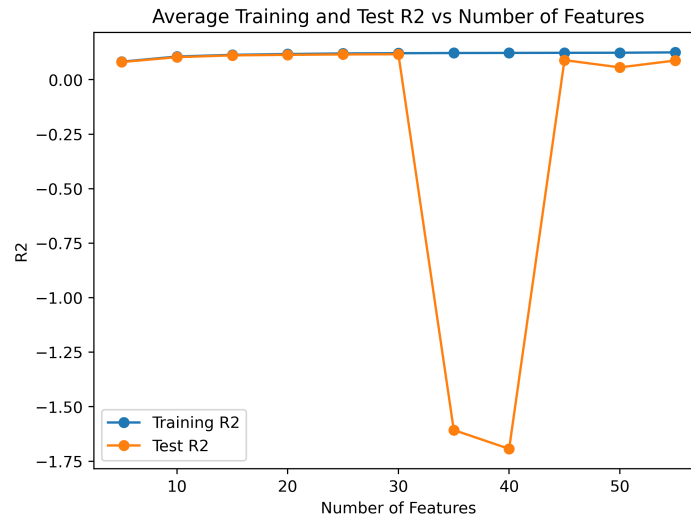


Figure 3: Average training and test R^2 vs k .

For R^2 to be negative, all we need is for $\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$ to be greater than 1. In other words, $\sum_{i=1}^n (y_i - \hat{y}_i)^2 > \sum_{i=1}^n (y_i - \bar{y})^2$, assuming $\sum_{i=1}^n (y_i - \bar{y})^2 > 0$, which should be appropriate if all y_i are not identical (aka if $\bar{y} = y_i \quad \forall i$).

To demonstrate this more concretely, what if we let all $\hat{y}$ be a constant c ? In that case:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - c)^2$$

and if we complete the square, we get:

$$\sum_{i=1}^n (y_i - c)^2 = \sum_{i=1}^n (y_i - \bar{y})^2 + n(\bar{y} - c)^2$$

plugging this into the formula for R^2 , we get:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \bar{y})^2 + n(\bar{y} - c)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = \frac{-n(\bar{y} - c)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

This is 0 if $c = \bar{y}$, negative for any other value of c , and there is no clear bound on how negative it can go. In other words, a negative R^2 value is possible if the predictions are far from $\bar{y}$.

- Random forest regression with 100 trees and maximum depths 1, 2, $\dots$, 10. Report the average training, OOB, and test R^2 and MSE and their standard deviations in each case. Also, plot the average training, OOB, and test R^2 vs the maximum depth as three separate curves.

In Python, we can do this and obtain table 3 and figure 4:

The average OOB R^2 almost exactly matches the test R^2

Depth	Train R2	Test R2	Train MSE	Test MSE	OOB R2	OOB MSE
1	0.055 (0.001)	0.054 (0.004)	0.818 (0.005)	0.819 (0.018)	0.054 (0.001)	0.819 (0.005)
2	0.082 (0.001)	0.079 (0.006)	0.795 (0.004)	0.797 (0.019)	0.079 (0.001)	0.797 (0.004)
3	0.104 (0.001)	0.099 (0.006)	0.775 (0.004)	0.78 (0.019)	0.099 (0.001)	0.78 (0.004)
4	0.124 (0.001)	0.114 (0.006)	0.758 (0.004)	0.767 (0.018)	0.113 (0.001)	0.768 (0.004)
5	0.145 (0.001)	0.124 (0.006)	0.741 (0.004)	0.758 (0.018)	0.124 (0.001)	0.759 (0.004)
6	0.17 (0.001)	0.132 (0.005)	0.718 (0.004)	0.751 (0.018)	0.131 (0.001)	0.752 (0.004)
7	0.203 (0.001)	0.139 (0.005)	0.69 (0.004)	0.746 (0.017)	0.137 (0.001)	0.747 (0.004)
8	0.244 (0.002)	0.143 (0.004)	0.654 (0.004)	0.742 (0.017)	0.142 (0.001)	0.743 (0.004)
9	0.293 (0.003)	0.147 (0.004)	0.612 (0.004)	0.738 (0.017)	0.144 (0.001)	0.741 (0.004)
10	0.348 (0.004)	0.15 (0.004)	0.564 (0.005)	0.736 (0.017)	0.146 (0.001)	0.739 (0.004)

Table 3: Mean (SD) of MSE and R^2 for various tree depths for a random forest of 100 trees.



Figure 4: Average Training, OOB, and Test R^2 vs Maximum Depth.

Code

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.ensemble import RandomForestRegressor
4 from sklearn.model_selection import KFold, cross_validate
5 from sklearn.dummy import DummyRegressor
6 import matplotlib.pyplot as plt
7 from sklearn.metrics import mean_squared_error, r2_score
8 from sklearn.linear_model import LinearRegression
9 from sklearn.feature_selection import SequentialFeatureSelector
10 import time
11
12 #reading in the csv file
13 df =
14     pd.read_csv('assignments/hw02/data/OnlineNewsPopularity/OnlineNewsPopularity.csv')
15
16 #dropping the first two columns by name. space in timedelta
17 columns_to_drop = ['url', 'timedelta']
18 df_new = df.drop(columns=columns_to_drop)
19
20 #selecting all but the last column as the predictors
21 X = df_new.iloc[:, :-1]
22
23 #selecting the last column as response
24 Y = np.log(df_new.iloc[:, -1])
25
26 #null model, simply using y_bar to predict y
27 #set up cross validation
28 cv = KFold(n_splits=5, random_state=1, shuffle=True)
29
30 #using DummyRegressor always predicting the mean mean of the training set
31 null_model = DummyRegressor(strategy="mean")
32
33 results = cross_validate(null_model, X=X, y=Y, cv=cv,
34     scoring="neg_mean_squared_error", return_train_score=True)
35
36 #cross validate uses the negative mean squared error. Need to negative the
37 negative
38 print("Train_MSE:", -results["train_score"].mean())
39 print("Test_MSE:", -results["test_score"].mean())
40
41 #OLS ridge regression
42 lambdas = [0, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1]
43 ridge_results = []
44
45 #need to loop over the lambdas
```

```
43 for lam in lambdas:
44     for train_idx, test_idx in cv.split(X):
45         X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
46         y_train, y_test = Y.iloc[train_idx], Y.iloc[test_idx]
47
48         #need to add a column on ones
49         X_train = np.column_stack([np.ones(len(X_train)), X_train])
50         X_test = np.column_stack([np.ones(len(X_test)), X_test])
51
52         #need to get N and p
53         N, p = X_train.shape
54
55         #use @ for matrix multiplication
56         X_crossprod_N = (X_train.T @ X_train)/N
57         X_cross_Y_N = (X_train.T @ y_train)/N
58
59         #LHS. Don't penalize the intercept
60         P = (lam*np.eye(p))
61         P[0, 0] = 0.0
62
63         LHS = X_crossprod_N + P
64
65         #solve
66         #https://numpy.org/devdocs/reference/generated/numpy.linalg.solve.html
67         beta_hat = np.linalg.solve(LHS, X_cross_Y_N)
68
69         #make predictions
70         y_train_predictions = X_train @ beta_hat
71         y_test_predictions = X_test @ beta_hat
72
73         #log of determinant of X_crossprod_N + (lambda*I)
74         #https://numpy.org/devdocs/reference/generated/numpy.linalg.slogdet.html
75         logdet = np.linalg.slogdet(LHS)[1]
76
77         #add this stuff to the ridge results
78         ridge_results.append({
79             "lambda": lam,
80             "train_mse": mean_squared_error(y_train, y_train_predictions),
81             "train_R2": r2_score(y_train, y_train_predictions),
82             "test_mse": mean_squared_error(y_test, y_test_predictions),
83             "test_R2": r2_score(y_test, y_test_predictions),
84             "logdeterm": logdet
85         })
86
87 ridge_results = pd.DataFrame(ridge_results)
88
89 #need to average across the 5 folds
```

```

90 ridge_results = ridge_results.groupby("lambda").agg(
91     train_mse_mean=("train_mse", "mean"),
92     train_mse_sd=("train_mse", "std"),
93     train_R2_mean=("train_R2", "mean"),
94     train_R2_sd=("train_R2", "std"),
95     test_mse_mean=("test_mse", "mean"),
96     test_mse_sd=("test_mse", "std"),
97     test_R2_mean=("test_R2", "mean"),
98     test_R2_sd=("test_R2", "std"),
99     logdeterm_mean=("logdeterm", "mean")
100 )
101
102 #need this as a latex table
103 formatted_summary = pd.DataFrame({
104     "Lambda": ridge_results.index,
105
106     "Train_R2": (
107         ridge_results["train_R2_mean"].round(3).astype(str)
108         + "\u2193("
109         + ridge_results["train_R2_sd"].round(3).astype(str)
110         + ")"
111     ),
112
113     "Test_R2": (
114         ridge_results["test_R2_mean"].round(3).astype(str)
115         + "\u2193("
116         + ridge_results["test_R2_sd"].round(3).astype(str)
117         + ")"
118     ),
119
120     "Train_MSE": (
121         ridge_results["train_mse_mean"].round(3).astype(str)
122         + "\u2193("
123         + ridge_results["train_mse_sd"].round(3).astype(str)
124         + ")"
125     ),
126
127     "Test_MSE": (
128         ridge_results["test_mse_mean"].round(3).astype(str)
129         + "\u2193("
130         + ridge_results["test_mse_sd"].round(3).astype(str)
131         + ")"
132     )
133 })
134
135 #format for latex
136 #using the column format option felt pretty slick

```

```

137 #need escape bc of special characters for LATEX
138 LATEX_table = formatted_summary.to_latex(
139     index=False,
140     caption="Mean (SD) of MSE and  $R^2$  5-fold CV. Ridge Like Regression.",
141     label="tab:ridge_mean_sd",
142     column_format="c" * formatted_summary.shape[1],
143     escape=False
144 )
145
146 print(LATEX_table)
147
148 #plot of average training and test MSE vs Lambda
149 plt.figure()
150 plt.plot(ridge_results.index.to_numpy(),
151          ridge_results["train_mse_mean"].to_numpy(), label="Training MSE")
152 plt.plot(ridge_results.index.to_numpy(),
153          ridge_results["test_mse_mean"].to_numpy(), label="Test MSE")
154 plt.xscale("log")
155 plt.xlabel("Lambda")
156 plt.ylabel("Average MSE")
157 plt.title("Average Training and Test MSE vs Lambda")
158 plt.legend()
159 plt.tight_layout()
160 #plt.show()
161 plt.savefig("assignments/hw02/figures/train_test_mse.png", dpi=400,
162            bbox_inches="tight")
163 plt.close()
164
165 #plot of average log determinant vs lambda
166 plt.figure()
167 plt.plot(ridge_results.index.to_numpy(),
168          ridge_results["logdeterm_mean"].to_numpy(), marker="o")
169 plt.xscale("log")
170 plt.xlabel("Lambda")
171 plt.ylabel("Average determinant")
172 plt.title("Average Log Determinent vs Lambda")
173 plt.tight_layout()
174 #plt.show()
175 plt.savefig("assignments/hw02/figures/log_determinant.png", dpi=400,
176            bbox_inches="tight")
177 plt.close()
178
179 #proof of concept from sklearn documentation
180 #for i, (train_index, test_index) in enumerate(cv.split(X)):
181     #print(f"Fold {i}:")
182     #print(f"  Train: index={train_index}")
183     #print(f"  Test:  index={test_index}")

```


179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225

```
#OLS regression with backwards elimination
#https://scikit-learn.org/stable/modules/generated/sklearn.feature\_selection.SequentialFeatureSelector.html
#note on r2 score: It can be negative:
#https://scikit-learn.org/stable/modules/generated/sklearn.metrics.r2\_score.html#sklearn.metrics.r2\_score

k_values = range(5, 60, 5)
results = []

for k in k_values:
    print(k)

    training_mse, testing_mse = [], []
    training_R2, testing_R2 = [], []

    #outer cv split
    for fold, (train_idx, test_idx) in enumerate(cv.split(X), start=1):
        print(f"Fold {fold}/{cv.get_n_splits()}")

        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = Y.iloc[train_idx], Y.iloc[test_idx]

        #feature selection. What score to use?
        #R uses R2 by default, so let's do the same
        #maybe try using n_jobs?
        feat_select = SequentialFeatureSelector(
            LinearRegression(),
            n_features_to_select=k,
            direction="backward",
            scoring="r2",
            cv=None,
            n_jobs=-1
        )

        t0 = time.time()
        feat_select.fit(X_train, y_train)
        print(f"Time: {time.time() - t0:.3f}s, fold: {fold}")

        #model_fitted = feat_select.fit(X_train, y_train)

        #selected features
        train_selected = feat_select.transform(X_train)
        test_selected = feat_select.transform(X_test)

        #fit
        model_selected = LinearRegression()
```

```
226     model_selected.fit(train_selected, y_train)
227
228     y_train_predictions = model_selected.predict(train_selected)
229     y_test_predictions = model_selected.predict(test_selected)
230
231     training_R2.append(r2_score(y_train, y_train_predictions))
232     testing_R2.append(r2_score(y_test, y_test_predictions))
233
234     training_mse.append(mean_squared_error(y_train, y_train_predictions))
235     testing_mse.append(mean_squared_error(y_test, y_test_predictions))
236
237     #store things again
238     results.append({
239         "k": k,
240         "train_r2": np.mean(training_R2),
241         "test_r2": np.mean(testing_R2),
242         "train_mse": np.mean(training_mse),
243         "test_mse": np.mean(testing_mse)
244     })
245
246 results_df = pd.DataFrame(results)
247 print(results_df)
248
249 #latex table. no index. only 4 decimal places
250 LATEX_table = results_df[["k", "train_r2", "test_r2"]].to_latex(
251     index=False,
252     float_format="%.4f",
253     caption="Average Training and Test  $R^2$  for Backward SFS",
254     label="tab:r2_feature_selection",
255     column_format="ccc"
256 )
257
258 print(LATEX_table)
259
260 #need to plot
261 plt.figure()
262 plt.plot(results_df["k"].to_numpy(), results_df["train_r2"].to_numpy(),
263         marker="o", label="Training R2")
264 plt.plot(results_df["k"].to_numpy(), results_df["test_r2"].to_numpy(),
265         marker="o", label="Test R2")
266 plt.xlabel("Number of Features")
267 plt.ylabel("R2")
268 plt.title("Average Training and Test R2 vs Number of Features")
269 plt.legend()
270 plt.tight_layout()
271 #plt.show()
```

```
270 plt.savefig("assignments/hw02/figures/feature_selection_train_test.png",
271             dpi=400, bbox_inches="tight")
272 plt.close()
273
274 #random forest regression
275 #need to change back to 11
276 depths = range(1, 11)
277 rows = []
278
279 #loop over depths
280 for d in depths:
281     print(d)
282     #getting idx of the train and test amongst the splits
283     for train_idx, test_idx in cv.split(X):
284         X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
285         y_train, y_test = Y.iloc[train_idx], Y.iloc[test_idx]
286
287         #turning on the oob_score
288         rf = RandomForestRegressor(n_estimators=100, max_depth=d,
289                                   oob_score=True, random_state=1234)
289
290         #fitting stuff
291         rf.fit(X_train, y_train)
292
293         #make predictions
294         y_pred_train = rf.predict(X_train)
295         y_pred_test = rf.predict(X_test)
296
297         #results
298         #using the oob_prediction_ to compute MSE
299         results = {
300             "depth": d,
301             "train_mse": mean_squared_error(y_train, y_pred_train),
302             "train_R2": r2_score(y_train, y_pred_train),
303             "oob_R2": rf.oob_score_,
304             "oob_MSE": mean_squared_error(y_train, rf.oob_prediction_),
305             "test_mse": mean_squared_error(y_test, y_pred_test),
306             "test_R2": r2_score(y_test, y_pred_test)
307         }
308
309         rows.append(results)
310
311 #making table and plotting
312 table1 = pd.DataFrame(rows)
313
314 #need to get means and standard deviations
```

```
315 #using agg function
316 summary_table = table1.groupby("depth").agg(
317     train_r2_mean=("train_R2", "mean"),
318     train_r2_sd=("train_R2", "std"),
319     oob_r2_mean=("oob_R2", "mean"),
320     oob_r2_sd=("oob_R2", "std"),
321     test_r2_mean=("test_R2", "mean"),
322     test_r2_sd=("test_R2", "std"),
323     train_mse_mean=("train_mse", "mean"),
324     train_mse_sd=("train_mse", "std"),
325     oob_mse_mean=("oob_MSE", "mean"),
326     oob_mse_sd=("oob_MSE", "std"),
327     test_mse_mean=("test_mse", "mean"),
328     test_mse_sd=("test_mse", "std"),
329 )
330
331 #better way to report this?
332 #need astype(str). Rounding also probably helps
333
334 formatted_summary = pd.DataFrame({
335     "Lambda": summary_table.index,
336
337     "Train_R2": (
338         summary_table["train_r2_mean"].round(3).astype(str)
339         + " ("
340         + summary_table["train_r2_sd"].round(3).astype(str)
341         + ")"
342     ),
343
344     "Test_R2": (
345         summary_table["test_r2_mean"].round(3).astype(str)
346         + " ("
347         + summary_table["test_r2_sd"].round(3).astype(str)
348         + ")"
349     ),
350
351     "Train_MSE": (
352         summary_table["train_mse_mean"].round(3).astype(str)
353         + " ("
354         + summary_table["train_mse_sd"].round(3).astype(str)
355         + ")"
356     ),
357
358     "Test_MSE": (
359         summary_table["test_mse_mean"].round(3).astype(str)
360         + " ("
361         + summary_table["test_mse_sd"].round(3).astype(str)
```

```

362         + ")"
363     ),
364
365     "OOB_R2": (
366         summary_table["oob_r2_mean"].round(3).astype(str)
367         + "\n("
368         + summary_table["oob_r2_sd"].round(3).astype(str)
369         + ")"
370     ),
371
372     "OOB_MSE": (
373         summary_table["oob_mse_mean"].round(3).astype(str)
374         + "\n("
375         + summary_table["oob_mse_sd"].round(3).astype(str)
376         + ")"
377     )
378 })
379
380 print(formatted_summary)
381
382 #format for latex
383 #using the column format option felt pretty slick
384 #need escape bc of special characters for LATEX
385 LATEX_table = formatted_summary.to_latex(
386     index=False,
387     caption="Mean (SD) of MSE and  $R^2$  for various tree depths for a random
388     forest of 100 trees.",
389     label="tab:mean_sd_rf",
390     column_format="c" * formatted_summary.shape[1],
391     escape=False
392 )
393
394 print(LATEX_table)
395
396 #needed to convert to np array
397 plt.figure()
398 plt.plot(summary_table.index.to_numpy(),
399          summary_table["train_r2_mean"].to_numpy(), label="Training_R2")
400
401 plt.plot(summary_table.index.to_numpy(),
402          summary_table["oob_r2_mean"].to_numpy(), label="OOB_R2")
403
404 plt.plot(summary_table.index.to_numpy(),
405          summary_table["test_r2_mean"].to_numpy(), label="Test_R2")
406
407 plt.xlabel("Maximum_Tree_Depth")
408 plt.ylabel("Average_R2")
409 plt.title("Average Training, OOB, and Test R2 vs Tree Depth")
410 plt.legend()

```

```
405 plt.tight_layout()
406 #plt.show()
407 plt.savefig("assignments/hw02/figures/train_oob_test_r2.png", dpi=400,
408             bbox_inches="tight")
plt.close()
```